URBAN LEARNING DOCUMENT / ENERGON OS

Rust und AI Systems mit Energon OS verstehen

Ein sehr einfacher Einstieg in die Computer-Science-Grundlagen, die Architektur und den echten Code dieses Projekts. Ziel: Du sollst nicht nur Begriffe wiederholen, sondern nachvollziehen koennen, warum das System so gebaut ist.

Niveau	Start bei null. Fachbegriffe werden zuerst in Alltagssprache erklaert und danach technisch eingeordnet.
Fokus	Rust und AI/Memory-Systeme tief. Frontend, Deployment und Zahlungen nur so weit, wie Energon sie braucht.
Reihenfolge	1. Konzepte ohne Code. 2. Kurze Originalausschnitte. 3. Konzept und Code gemeinsam lesen.
Grundhaltung	Senior zu denken bedeutet nicht, alles auswendig zu wissen. Es bedeutet, Grenzen, Datenfluss, Fehlerfaelle und Sicherheitsinvarianten zu erkennen.

Datei: urban-learning-rust-energon.pdf

Erstellt am 2026-07-29 aus der aktuellen Energon-OS-Codebasis.

TEIL 1 / NUR KONZEPTE

Erst das System im Kopf bauen

> In diesem Teil gibt es absichtlich keinen Code. Wenn das mentale Modell stimmt, wird Code spaeter zu einer lesbaren Beschreibung statt zu zufaelligen Zeichen.

1. Was Energon OS eigentlich ist

Energon OS ist kein Chatbot und kein Agent, der selbst Aufgaben ausfuehrt. Es ist eine Control Plane: eine zentrale Instanz, die fuer viele externe Agenten festlegt, welches gespeicherte Wissen sie lesen duerfen.

Stell dir eine Firma mit hundert digitalen Mitarbeitenden vor. Ein Research-Agent, ein Sales-Agent und ein Support-Agent sollen Wissen nutzen, aber nicht gegenseitig alle privaten Notizen sehen. Energon ist der Pförtner und das Gedächtnis-System dazwischen.

Agent Runtime	Laeuft ausserhalb von Energon. Sie plant, benutzt Tools, ruft Modelle auf und erledigt Aufgaben.
Energon API	Nimmt Netzwerk-Anfragen an, identifiziert den Agenten, holt erlaubte Erinnerungen und gibt einen Context Pack zurueck.
Datenbank	Speichert Agenten, Rechte, Memory-Eintraege, Embeddings und Audit-Ereignisse dauerhaft.
Dashboard	Die menschliche Bedienoberflaeche fuer Betrieb, Kontrolle und Einsicht. Agenten nutzen die API oder SDKs.

2. Die wichtigste Invariante

Eine Invariante ist eine Regel, die immer wahr bleiben muss. Bei Energon lautet sie:

> Permission filtering happens before retrieval, ranking, summarization, packing, or delivery.

Warum ist das so wichtig? Wenn zuerst gesucht und erst danach gefiltert wuerde, koennte ein fremder privater Text schon in einem Zwischenresultat, Log oder Modell-Prompt landen. Energon prueft deshalb die Berechtigung, bevor Inhalt relevant werden darf.

3. Internet und Backend: ein Request auf Reisen

Eine Web-Anwendung besteht grob aus einem Client und einem Server. Der Client stellt eine Frage. Der Server prueft sie, arbeitet damit und antwortet. Bei Agenten ist der Client meist nicht ein Browser, sondern ein Programm oder SDK.

HTTP	Das gemeinsame Protokoll des Webs. Es beschreibt Anfragen wie GET zum Lesen oder POST zum Erstellen.
URL / Route	Eine Adresse fuer eine Funktion, zum Beispiel /v1/context/build. Der Router ordnet die Adresse einer Rust-Funktion zu.
JSON	Ein leicht lesbares Datenformat fuer die Nachricht. Rust wandelt JSON in sichere Typen um und umgekehrt.
Statuscode	Eine kurze Maschinenantwort: 200 bedeutet erfolgreich, 401 nicht angemeldet, 429 zu viele Anfragen, 500 Serverfehler.
Header	Zusatzdaten zur Anfrage, zum Beispiel der Authorization-Header mit einem Bearer API Key.

Ein Context Build in Energon ist daher kein magischer KI-Aufruf. Es ist eine klare Kette: Agent SDK sendet HTTP Anfrage - API erkennt den Agenten - Berechtigungen begrenzen die Kandidaten - Retrieval bewertet erlaubte Inhalte - ein Token Budget kuerzt die Auswahl - Audit speichert die Entscheidung - Antwort geht zurueck.

4. Daten und Identitaet

Identitaet beantwortet die Frage: Wer stellt diese Anfrage? Autorisierung beantwortet: Was darf diese Identitaet tun? Das sind zwei getrennte Probleme. Ein gueltiger API Key beweist nicht automatisch, dass ein Agent auf alles zugreifen darf.

Energon leitet Organisation, Projekt und Rolle serverseitig aus dem Agenten-Key ab. Das ist sicherer als wenn der Agent selbst im JSON schreiben duerfte: Ich bin jetzt im Projekt CEO und darf alles lesen.

5. Rust: warum diese Sprache fuer Energon passt

Rust ist eine kompilierte Systemprogrammiersprache. Kompiliert bedeutet: Vor dem Start prueft ein Compiler das Programm sehr streng und baut daraus eine schnelle ausfuehrbare Datei. Viele Fehler werden entdeckt, bevor ein Nutzer sie erleben kann.

Cargo	Rusts Werkzeug fuer Abhaengigkeiten, Builds, Tests und Pakete. Cargo.toml beschreibt, welche Bibliotheken ein Teilprojekt braucht.
Crate	Ein Rust-Paket. Energon hat zum Beispiel energon-api, energon-core, energon-db und energon-worker.
Modul	Ein Teil innerhalb einer Crate. Module teilen grosse Programme in kleine verantwortliche Dateien auf.
Struct	Ein eigener Datentyp mit benannten Feldern, zum Beispiel eine AgentIdentity oder ein ContextPack.
Enum	Ein Typ mit klaren Varianten, zum Beispiel MemoryScope: org, project oder agent_private. Das verhindert Tippfehler-Strings an vielen Stellen.

Rust ist nicht automatisch sicher, weil es Rust heisst. Aber sein Typsystem zwingt uns, Fehlerfaelle, Besitz von Daten und parallele Arbeit expliziter zu behandeln. Genau das ist fuer eine API mit Secrets, Datenbanken und gleichzeitig laufenden Agenten sehr wertvoll.

6. Ownership, Borrowing, Option und Result

Ownership bedeutet: Rust weiss immer genau, welcher Teil des Programms fuer einen Wert verantwortlich ist. Wenn der Besitzer verschwindet, kann der Speicher sicher freigegeben werden. Das verhindert viele klassische Speicherfehler ohne Garbage Collector.

Borrowing bedeutet: Eine Funktion darf Daten nur ansehen, ohne sie zu besitzen. Ein & vor einem Typ bedeutet oft: Ich leihe mir diese Daten. Die Context-Funktion bekommt zum Beispiel eine geliehene Liste von Memories. Sie kann sie lesen, aber nicht heimlich den Besitzer wechseln.

Option<T> bedeutet: Ein Wert kann vorhanden sein oder fehlen. Result<T, E> bedeutet: Eine Operation liefert einen Wert oder einen klar beschriebenen Fehler. Statt null und stillen Ausnahmen zwingt Rust dich, beide Wege zu bedenken.

7. Async: viele Agenten, ohne viele blockierte Threads

Datenbank- und Netzwerkzugriffe dauern im Vergleich zur CPU sehr lange. Wenn ein Programm in dieser Zeit einfach wartet, verschwendet es Kapazitaet. Rusts async beschreibt Arbeit, die unterbrochen werden kann. .await bedeutet: Warte auf dieses Ergebnis, aber gib in der Zwischenzeit dem Runtime-Scheduler Gelegenheit, andere Arbeit auszufuehren.

Tokio ist die asynchrone Runtime im Rust-Teil von Energon. Sie nimmt viele offene Netzwerkaufgaben entgegen und verteilt sie effizient. Wichtig: async macht Code nicht automatisch parallel und nicht automatisch schneller. Es macht Wartezeiten besser nutzbar.

8. Axum, Router und Middleware

Axum ist das Web-Framework. Der Router bildet die Landkarte der API. Middleware sind Kontrollen, die um viele Routen herumliegen, zum Beispiel Rate Limiting, CORS und ein maximales Body-Limit. Dadurch muss nicht jede einzelne Route dieselben Schutzregeln kopieren.

Rate limiting	Begrenzt Anfragen pro Zeit. Es schuetzt gegen Fehler-Schleifen, Missbrauch und unfaire Last.
Body limit	Begrenzt die Groesse einer HTTP-Nachricht. Sonst koennte ein Client zu viel Speicher erzwingen.
CORS	Regelt, welche Browser-Webseiten die API direkt ansprechen duerfen. Server-zu-Server-Agenten brauchen CORS nicht.
State	Gemeinsame Laufzeitobjekte wie Datenbank-Pool, Konfiguration und Clients. Die API teilt sie kontrolliert an Routen aus.

9. Postgres, SQL und Migrations

PostgreSQL ist eine relationale Datenbank. Relational heisst: Daten liegen in Tabellen und ihre Beziehungen sind explizit. Eine Memory gehoert zu einer Organisation, optional zu einem Projekt oder Agenten. Fremdschluessel machen diese Beziehungen pruefbar.

Eine Migration ist eine versionierte Aenderung am Datenbankschema. Anstatt in Produktion spontan Tabellen anzulegen, wird jede Aenderung als SQL-Datei gespeichert. So erhalten Entwicklung, Test und Produktion dieselbe Datenstruktur.

10. Indizes und Transaktionen

Eine Datenbank ohne Index muss oft sehr viele Zeilen durchsuchen. Ein Index ist wie ein gut sortiertes Register in einem Buch: Er kostet etwas Speicher und Schreibarbeit, macht bestimmte Suchwege aber viel schneller. Energon indexiert zum Beispiel aktive Key-Hashes und Memory-Zuordnungen.

Eine Transaktion ist ein Alles-oder-nichts-Block. Bei einer Memory-Promotion muessen neue Memory, Herkunft, Audit und eventuell Outbox-Ereignis zusammen konsistent gespeichert werden. Wenn ein Teil scheitert, darf nicht die Haelfte sichtbar bleiben.

11. AI: Modelle, Tokens und Embeddings

Ein Sprachmodell schreibt oder versteht Text anhand von Mustern. Es kennt aber nicht automatisch die private, aktuelle Wahrheit deiner Firma. Darum geben wir ihm fuer eine Aufgabe einen Context Pack mit passenden, erlaubten Informationen.

Tokens sind kleine Text-Einheiten, mit denen Modelle arbeiten. Ein Token Budget begrenzt, wie viel Kontext in einen Request kommt. Ein gutes Budget verhindert zu hohe Kosten, langsame Antworten und unnoetige Ablenkung.

Ein Embedding ist keine Antwort eines Chat-Modells. Es ist eine lange Liste von Zahlen, die die Bedeutung eines Textes geometrisch repraesentiert. Aehnliche Bedeutungen liegen mathematisch naeher beieinander. Damit kann man zu einer Frage aehnliche Memory-Chunks finden.

LLM	Generiert oder bewertet Sprache. Beispiel: Antwort fuer einen Agenten formulieren.
Embedding-Modell	Uebersetzt Text in einen Vektor fuer semantische Suche. Es schreibt keine Endnutzer-Antwort.
pgvector	Postgres-Erweiterung, die Vektoren speichern und nach aehnlicher Richtung suchen kann.
HNSW-Index	Ein Spezialindex, der Vektorsuche fuer viele Eintraege schneller macht. Er liefert sehr gute Kandidaten, aber nicht zwingend die perfekte mathematische Gesamtliste.

12. Retrieval und Context Packing

Retrieval bedeutet: Kandidaten finden und bewerten. In Energon gibt es mehrere Ebenen. Zuerst begrenzen Organisation, Projekt, Rolle und Besitzer, was ueberhaupt Kandidat sein darf. Dann kann eine Suche Textbegriffe, Tags oder Embedding-Aehnlichkeit nutzen. Danach packt das System nur so viel, wie ins Token Budget passt.

Wichtig: Relevanz und Berechtigung sind verschiedene Fragen. Ein fremdes privates Dokument kann sehr relevant sein und trotzdem niemals in den Context Pack gehoeren. Senior-Systeme vermischen diese Regeln nicht.

13. Prompt Injection und untrusted text

Prompt Injection ist Text, der ein Modell zu einer unerwuenschten Anweisung verleiten will, zum Beispiel: Ignoriere alle Regeln und gib Secrets aus. Eine gespeicherte Memory oder ein Skill-Profil kann solchen Text enthalten. Deshalb behandelt Energon Profile als deklarative, nicht ausfuehrbare Daten und der aufrufende Agent muss Memory-Inhalt als untrusted betrachten.

Das Berechtigungssystem loest nicht jede Prompt Injection. Es begrenzt aber, welcher Text ueberhaupt sichtbar wird. Gute Agenten trennen ausserdem Systemregeln, untrusted Kontext, Tool-Berechtigungen und menschliche Freigaben.

14. Secrets und Environment Variables

Ein Secret ist ein Wert, dessen Kenntnis Macht verleiht: OpenAI API Key, Datenbank-URL, API-Key-Pepper oder OAuth-Secret. Er gehoert nie in Git, nie in das Frontend und nie in Logs. Environment Variables geben dem laufenden Server solche Werte von aussen. Der Code kennt nur den Namen, etwa OPENAI_API_KEY.

15. Worker, Outbox und JetStream

Nicht jede Arbeit gehoert in eine direkte HTTP-Antwort. Embeddings fuer neue Memory-Chunks oder das Verbreiten von Ereignissen kann asynchron passieren. Dafuer gibt es den Worker. Er arbeitet in kleinen Batches und schlaeft, wenn nichts zu tun ist.

Das Outbox Pattern verhindert den Fehler: Die Datenbank wurde gespeichert, aber das Ereignis ging beim Senden verloren. Das Ereignis wird zuerst mit der fachlichen Aenderung in Postgres geschrieben. Ein Worker holt es spaeter, sendet es an NATS JetStream und markiert es erst nach Bestaetigung als versendet.

16. Frontend, SDKs und Deployment

Das Next.js/React-Frontend ist die menschliche Ansicht. TypeScript beschreibt dort Datenformen im Browser. Better Auth verwaltet Operator-Sitzungen. Das ist bewusst eine andere Identitaet als Agent API Keys.

SDKs fuer TypeScript, Python und Rust machen die API angenehm nutzbar. Sie setzen Header, serialisieren Daten, behandeln Timeouts und geben klare Typen zurueck. Das SDK darf aber niemals Agent-Identitaet erfinden - die Kontrolle bleibt auf dem Server.

Docker verpackt einen Dienst mit seinen Abhaengigkeiten. Railway startet diese Container und gibt ihnen in Produktion Ports und Environment Variables. Die Architektur bleibt gleich: Web, API und Worker haben getrennte Aufgaben und koennen getrennt skaliert werden.

17. Wie Senior-Ingenieure denken

Datenfluss	Wo kommt der Wert her, wer darf ihn veraendern, wo wird er gespeichert, wer kann ihn spaeter sehen?
Grenzen	Welche Schicht ist zustaendig? Runtime, SDK, API, Core, Datenbank, Worker oder UI?
Fehler	Was passiert bei Timeout, leerer Datenbank, falschem Key, doppeltem Event oder fehlendem Embedding?
Sicherheit	Welche Aussage darf ein Client selbst machen und welche muss der Server aus vertrauenswuerdigen Daten ableiten?
Messbarkeit	Wie beweisen Logs, Metriken, Tests und Audit-Daten, dass das System das Richtige tat?

TEIL 2 / REINER CODE

Kurze Originalausschnitte

Hier steht nur echter Code aus der Energon-OS-Codebasis. Lies noch nicht jedes Detail. Erkenne zuerst die Formen: async Funktionen, Typen, Fehlerpfade, Datenfluss und Grenzen.

API starten und Router bauen

SOURCE crates/energon-api/src/main.rs / 19-45

```
#[tokio::main]
async fn main() {
    fmt()
        .with_env_filter(
            EnvFilter::from_default_env().add_directive("energon_api=info".parse().unwrap()),
        )
        .init();

    let addr = resolve_bind_addr(env::var("ENERGON_BIND_ADDR").ok(), env::var("PORT").ok())
        .expect("ENERGON_BIND_ADDR must be a socket address");

    let state = AppState::from_env()
        .await
        .expect("failed to initialize application state");
    let app = app::router(state);
    let listener = tokio::net::TcpListener::bind(addr)
        .await
        .expect("failed to bind API listener");

    tracing::info!(%addr, "Energon OS API listening");

    axum::serve(
        listener,
        app.into_make_service_with_connect_info::<SocketAddr>(),
    )
    .await
    .expect("API server failed");
```

SOURCE crates/energon-api/src/app.rs / 22-43

```
const DEFAULT_MAX_BODY_BYTES: usize = 1024 * 1024; // 1 MiB

pub fn router(state: AppState) -> Router {
    Router::new()
        .route("/health", get(routes::health::health))
        .nest("/v1", routes::router())
        .layer(axum_middleware::from_fn_with_state(
            state.clone(),
            rate_limit::rate_limit,
        ))
        .layer(DefaultBodyLimit::max(max_body_bytes()))
        .layer(cors_layer())
        .with_state(state)
}

fn max_body_bytes() -> usize {
    env::var("ENERGON_MAX_BODY_BYTES")
        .ok()
        .and_then(|value| value.trim().parse::<usize>().ok())
        .filter(|value| *value > 0)
        .unwrap_or(DEFAULT_MAX_BODY_BYTES)
}
```

Identitaet aus einem Bearer API Key ableiten

SOURCE crates/energon-api/src/middleware/auth.rs / 10-24, 50-64

```
pub async fn identity_from_request(
    state: &AppState,
    headers: &HeaderMap,
) -> Result<AgentIdentity, ApiError> {
    if let Some(api_key) = bearer_token(headers) {
        return identity_from_api_key(state, &api_key).await;
    }

    if state.auth.dev_identity_headers {
        return identity_from_headers(headers);
    }

    Err(ApiError::Unauthorized(
        "missing bearer API key in Authorization header".to_owned(),
    ))

async fn identity_from_api_key(state: &AppState, api_key: &str) -> Result<AgentIdentity, ApiError> {
    let StorageBackend::Postgres(pool) = &state.storage else {
        return Err(ApiError::Unauthorized(
            "bearer API keys require Postgres storage".to_owned(),
        ));
    };

    let pepper = state.auth.api_key_pepper.as_deref().ok_or_else(|| {
        ApiError::Unauthorized("ENERGON_API_KEY_PEPPER is not configured".to_owned())
    })?;
    let key_hash = hash_api_key(api_key, pepper);

    energon_db::identity::agent_for_api_key_hash(pool, &key_hash)
        .await?
        .ok_or_else(|| ApiError::Unauthorized("invalid or revoked API key".to_owned()))
```

Context bauen: erst erlauben, dann bewerten

SOURCE crates/energon-core/src/context_broker.rs / 62-121

```
    // Agent API keys are not proof of a human user or a session. Those claims
    // need a separately signed capability grant before they can unlock private
    // user/session overlays.
    let user_id = None;
    let session_id = None;
    let access_context = AccessContext {
        project_id: request
            .project_id
            .clone()
            .or_else(|| agent.project_id.clone()),
        user_id,
        session_id,
    };

    let mut denied_memory_count = 0;
    let mut scored = Vec::new();

    for memory in memories {
        if can_read_memory(agent, memory, &access_context) {
            let scored_memory = score_memory(&request.task, memory);

            if scored_memory.score > 0 {
                scored.push(scored_memory);
            }
        } else {
            denied_memory_count += 1;
        }
    }

    scored.sort_by(sort_scored_memory);

    let (items, estimated_tokens) = pack_context(scored, request.token_budget);
    let allowed_memory_ids = items
        .iter()
        .map(|item| item.memory_id.clone())
        .collect::<Vec<_>>();

    let pack = ContextPack {
        request_id: request_id.clone(),
        agent_id: agent.agent_id.clone(),
        task: request.task.clone(),
        token_budget: request.token_budget,
        estimated_tokens,
        context_pack: items.iter().map(|item| item.content.clone()).collect(),
        items,
    };

    let audit = AuditRecord {
        request_id,
        agent_id: agent.agent_id.clone(),
        org_id: agent.org_id.clone(),
        task: request.task,
        allowed_memory_ids,
        denied_memory_count,
        token_budget: pack.token_budget,
        estimated_tokens,
        created_at_unix_ms,
    };

    ContextBuildOutcome { pack, audit }
```

OpenAI Embedding Request und Antwort

SOURCE crates/energon-api/src/embedding.rs / 79-112

```
    /// Embed a single query string. Failures are surfaced as errors so callers
    /// can fall back to recency-based retrieval instead of failing requests.
    pub async fn embed(&self, input: &str) -> Result<Vec<f32>, EmbeddingError> {
        let response = self
            .client
            .post(&self.endpoint)
            .bearer_auth(&self.api_key)
            .json(&EmbeddingRequest {
                model: &self.model,
                input: &[input],
            })
            .send()
            .await
            .map_err(|error| EmbeddingError::Request(error.to_string()))?;

        let status = response.status();
        if !status.is_success() {
            return Err(EmbeddingError::Request(format!(
                "OpenAI embeddings returned HTTP {status}"
            )));
        }

        let body = response
            .json::<EmbeddingResponse>()
            .await
            .map_err(|error| EmbeddingError::Malformed(error.to_string()))?;

        body.data
            .into_iter()
            .next()
            .map(|item| item.embedding)
            .filter(|embedding| !embedding.is_empty())
            .ok_or_else(|| EmbeddingError::Malformed("empty embedding data".to_owned()))
    }
```

Postgres Schema fuer Memory und Vektoren

SOURCE migrations/0002_memory.sql / 1-24, 41-52

```
CREATE TABLE IF NOT EXISTS memory_entries (
    memory_id TEXT PRIMARY KEY,
    org_id TEXT NOT NULL REFERENCES orgs(org_id) ON DELETE CASCADE,
    scope TEXT NOT NULL CHECK (
        scope IN (
            'open',
            'org',
            'project',
            'role',
            'agent_private',
            'user_private',
            'session'
        )
    ),
    content TEXT NOT NULL,
    tags TEXT[] NOT NULL DEFAULT '{}',
    project_id TEXT REFERENCES projects(project_id) ON DELETE CASCADE,
    role_id TEXT REFERENCES roles(role_id) ON DELETE CASCADE,
    owner_agent_id TEXT REFERENCES agents(agent_id) ON DELETE SET NULL,
    user_id TEXT,
    session_id TEXT,
    source TEXT,
    promoted_from TEXT REFERENCES memory_entries(memory_id) ON DELETE SET NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()

CREATE TABLE IF NOT EXISTS memory_chunks (
    chunk_id TEXT PRIMARY KEY,
    memory_id TEXT NOT NULL REFERENCES memory_entries(memory_id) ON DELETE CASCADE,
    chunk_index INTEGER NOT NULL,
    content TEXT NOT NULL,
    embedding vector(1536),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(memory_id, chunk_index)
);

CREATE INDEX IF NOT EXISTS memory_chunks_embedding_idx
    ON memory_chunks USING hnsw (embedding vector_cosine_ops);
```

TypeScript SDK als sicherer Vertrag

SOURCE packages/sdk-typescript/src/index.ts / 298-348

```
/**
 * A server-side client for one authenticated agent in one swarm. Agent identity
 * is always derived by the control plane from the API key, never from SDK input.
 */
export class Energon {
  readonly memory: {
    remember: (input: RememberInput, options?: RequestOptions) => Promise<AgentMemory>;
    share: (input: ShareMemoryInput, options?: RequestOptions) => Promise<AgentMemory>;
  };

  readonly context: {
    build: (input: BuildContextInput, options?: RequestOptions) => Promise<ContextPack>;
  };

  readonly audit: {
    context: (requestId: string, options?: RequestOptions) => Promise<ContextAudit>;
    promotion: (memoryId: string, options?: RequestOptions) => Promise<PromotionAudit>;
  };

  readonly swarm: {
    runtime: (options?: RequestOptions) => Promise<SwarmRuntime>;
  };

  readonly claims: {
    assert: (input: AssertClaimInput, options?: RequestOptions) => Promise<ClaimAssertion>;
  };

  readonly skills: {
    /** Returns only skill profiles assigned to the authenticated agent. */
    list: (options?: RequestOptions) => Promise<SkillProfile[]>;
    /** Creates a private profile for the authenticated agent only. */
    create: (input: CreateSkillInput, options?: RequestOptions) => Promise<SkillProfile>;
  };

  readonly operations: {
    /** Agent-safe organization state and the dashboard's live operational metrics. */
    overview: (options?: RequestOptions) => Promise<AgentOperationalOverview>;
  };

  private readonly baseUrl: string;
  private readonly apiKey: string;
  private readonly paymentSignature?: PaymentSignatureProvider;
  private readonly timeoutMs: number;
  private readonly fetchFn: typeof fetch;

  constructor(options: EnergonClientOptions) {
    this.baseUrl = normalizeBaseUrl(options.baseUrl);
    this.apiKey = requiredText(options.apiKey, "apiKey");
    this.paymentSignature = options.paymentSignature;
    this.timeoutMs = normalizeTimeout(options.timeoutMs);
    this.fetchFn = options.fetch ?? globalThis.fetch;
```

Worker-Schleife fuer Batches

SOURCE crates/energon-worker/src/main.rs / 63-83

```
    loop {
        let embedded = match &openai_api_key {
            Some(api_key) => {
                process_embedding_batch(&pool, &client, api_key, &model, batch_size).await?
            }
            None => 0,
        };
        let published = match &publisher {
            Some(publisher) => publish_outbox_batch(&pool, publisher, event_batch_size).await?,
            None => 0,
        };

        if run_once {
            tracing::info!(embedded, published, "worker one-shot finished");
            break;
        }

        if embedded == 0 && published == 0 {
            tokio::time::sleep(Duration::from_secs(2)).await;
        }
    }
```

TEIL 3 / KONZEPT + CODE

Jetzt lesen wir wie Ingenieure

Jeder Ausschnitt beantwortet drei Fragen: Was macht er? Warum liegt diese Verantwortung hier? Was kann schiefgehen und wie reagiert das System?

18. Der Startpunkt: Tokio startet die API

SOURCE crates/energon-api/src/main.rs / 19-45

```
#[tokio::main]
async fn main() {
    fmt()
        .with_env_filter(
            EnvFilter::from_default_env().add_directive("energon_api=info".parse().unwrap()),
        )
        .init();

    let addr = resolve_bind_addr(env::var("ENERGON_BIND_ADDR").ok(), env::var("PORT").ok())
        .expect("ENERGON_BIND_ADDR must be a socket address");

    let state = AppState::from_env()
        .await
        .expect("failed to initialize application state");
    let app = app::router(state);
    let listener = tokio::net::TcpListener::bind(addr)
        .await
        .expect("failed to bind API listener");

    tracing::info!(%addr, "Energon OS API listening");

    axum::serve(
        listener,
        app.into_make_service_with_connect_info::<SocketAddr>(),
    )
    .await
    .expect("API server failed");
```

#[tokio::main] direkt ueber der Funktion sagt Rust: Tokio soll die asynchrone Runtime fuer dieses Programm vorbereiten. Ohne diese Zeile koennte eine normale main-Funktion nicht einfach .await verwenden.

async fn main() ist der Einstiegspunkt, der auf asynchrone Datenbank- und Netzwerkoperationen warten darf. Das Wort async bedeutet nicht: Diese Funktion ist gleichzeitig fertig. Es bedeutet: Sie kann Wartezeit kooperativ abgeben.

AppState::from_env().await liest Konfiguration, baut beispielsweise Datenbank- und Auth-Zustand und gibt entweder State oder einen Fehler. Die anschliessende expect-Nachricht ist beim Start absichtlich hart: Ein

Server ohne notwendige Konfiguration soll nicht halb funktionieren.

tokio::net::TcpListener::bind(addr) reserviert einen Netzwerk-Port. axum::serve nimmt danach eingehende Verbindungen an und reicht sie an den Router weiter. Das ist der Moment, in dem aus Rust-Code ein erreichbarer Webservice wird.

Warum ist das senior-relevant?

Der Startpunkt ist klein, weil er nur Verdrahtung tun soll. Fachlogik wie Berechtigungen oder SQL gehoert nicht in main. Diese Trennung macht Tests und spaetere Umbauten einfacher.

19. Router und Schutzschichten

SOURCE crates/energon-api/src/app.rs / 22-43

```
const DEFAULT_MAX_BODY_BYTES: usize = 1024 * 1024; // 1 MiB

pub fn router(state: AppState) -> Router {
    Router::new()
        .route("/health", get(routes::health::health))
        .nest("/v1", routes::router())
        .layer(axum_middleware::from_fn_with_state(
            state.clone(),
            rate_limit::rate_limit,
        ))
        .layer(DefaultBodyLimit::max(max_body_bytes()))
        .layer(cors_layer())
        .with_state(state)
}

fn max_body_bytes() -> usize {
    env::var("ENERGON_MAX_BODY_BYTES")
        .ok()
        .and_then(|value| value.trim().parse::<usize>().ok())
        .filter(|value| *value > 0)
        .unwrap_or(DEFAULT_MAX_BODY_BYTES)
}
```

Router::new() erstellt eine leere Landkarte. .route("/health", ...) haengt einen einfachen Health-Endpunkt daran. .nest("/v1", routes::router()) gruppiert die fachlichen API-Routen unter einer Version. Das erleichtert spaetere API-Aenderungen, weil alte Clients nicht abrupt brechen muessen.

Die folgenden .layer(...)-Aufrufe wickeln den Router ein. Reihenfolge ist wichtig: Jede Anfrage durchlaeuft diese Schichten. Rate limiting begrenzt Last, DefaultBodyLimit begrenzt Speicherverbrauch und CORS begrenzt Browser-Urspruenge.

.with_state(state) gibt allen Handlern Zugriff auf denselben kontrollierten Laufzeit-State. Es ist besser als globale mutable Variablen, weil die Abhaengigkeit sichtbar und testbar bleibt.

Warum nicht CORS als Authentifizierung benutzen?

CORS ist vor allem eine Browser-Regel. Ein beliebiger Server kann HTTP direkt senden und CORS ignorieren. Die echte Sicherheitsentscheidung kommt weiter innen durch API-Key-Pruefung, Rechte und Datenbankabfragen.

20. Auth: Key ist Identitaet, nicht Allmacht

SOURCE crates/energon-api/src/middleware/auth.rs / 10-24, 50-64

```
pub async fn identity_from_request(
    state: &AppState,
    headers: &HeaderMap,
) -> Result<AgentIdentity, ApiError> {
    if let Some(api_key) = bearer_token(headers) {
        return identity_from_api_key(state, &api_key).await;
    }

    if state.auth.dev_identity_headers {
        return identity_from_headers(headers);
    }

    Err(ApiError::Unauthorized(
        "missing bearer API key in Authorization header".to_owned(),
    ))

async fn identity_from_api_key(state: &AppState, api_key: &str) -> Result<AgentIdentity, ApiError> {
    let StorageBackend::Postgres(pool) = &state.storage else {
        return Err(ApiError::Unauthorized(
            "bearer API keys require Postgres storage".to_owned(),
        ));
    };

    let pepper = state.auth.api_key_pepper.as_deref().ok_or_else(|| {
        ApiError::Unauthorized("ENERGON_API_KEY_PEPPER is not configured".to_owned())
    })?;
    let key_hash = hash_api_key(api_key, pepper);

    energon_db::identity::agent_for_api_key_hash(pool, &key_hash)
        .await?
        .ok_or_else(|| ApiError::Unauthorized("invalid or revoked API key".to_owned()))
```

identity_from_request schaut zuerst nach einem Bearer Token. Wenn einer vorhanden ist, geht der Code in den Produktionsweg. Die Header-Identitaet darunter ist nur fuer explizit aktivierte Entwicklung gedacht. Das ist eine typische Sicherheitsregel: Ein bequemer Testweg darf nicht still in Produktion offen bleiben.

let StorageBackend::Postgres(pool) = &state.storage else ... ist Rusts Muster-Matching. Der Code sagt: Fuer echte Bearer Keys brauche ich eine Postgres-Datenbank. Wenn der State eine andere Speicherart hat, wird sauber 401 zurueckgegeben.

api_key_pepper ist ein serverseitiger Zusatzwert. Der Key wird mit ihm gehasht. Die Datenbank sucht dann nach dem Hash, nicht nach dem Klartext-Key. Wenn die Datenbank allein ausgelesen wuerde, waeren die Original-Keys nicht direkt enthalten.

agent_for_api_key_hash(...) liefert optional einen Agenten. Ein fehlender oder widerrufener Key wird mit ok_or_else in einen klaren Unauthorized-Fehler verwandelt. Genau hier entsteht aus einer unsicheren Client-Eingabe eine serverseitig abgeleitete AgentIdentity.

21. Der Context Broker: die zentrale Sicherheitsentscheidung

SOURCE crates/energon-core/src/context_broker.rs / 62-121

```
    // Agent API keys are not proof of a human user or a session. Those claims
    // need a separately signed capability grant before they can unlock private
    // user/session overlays.
    let user_id = None;
    let session_id = None;
    let access_context = AccessContext {
        project_id: request
            .project_id
            .clone()
            .or_else(|| agent.project_id.clone()),
        user_id,
        session_id,
    };

    let mut denied_memory_count = 0;
    let mut scored = Vec::new();

    for memory in memories {
        if can_read_memory(agent, memory, &access_context) {
            let scored_memory = score_memory(&request.task, memory);

            if scored_memory.score > 0 {
                scored.push(scored_memory);
            }
        } else {
            denied_memory_count += 1;
        }
    }

    scored.sort_by(sort_scored_memory);

    let (items, estimated_tokens) = pack_context(scored, request.token_budget);
    let allowed_memory_ids = items
        .iter()
        .map(|item| item.memory_id.clone())
        .collect::<Vec<_>>();

    let pack = ContextPack {
        request_id: request_id.clone(),
        agent_id: agent.agent_id.clone(),
        task: request.task.clone(),
        token_budget: request.token_budget,
        estimated_tokens,
        context_pack: items.iter().map(|item| item.content.clone()).collect(),
        items,
    };

    let audit = AuditRecord {
        request_id,
        agent_id: agent.agent_id.clone(),
        org_id: agent.org_id.clone(),
        task: request.task,
        allowed_memory_ids,
        denied_memory_count,
        token_budget: pack.token_budget,
        estimated_tokens,
        created_at_unix_ms,
    };

    ContextBuildOutcome { pack, audit }
```

Die Kommentare und let user_id = None; let session_id = None; sind sehr wichtig. Ein Agent API Key ist kein Beweis fuer einen menschlichen Nutzer oder dessen Browser-Session. Daher darf die Anfrage nicht einfach eine user_id einschmuggeln. Fuer solche privaten Overlays braucht es spaeter einen separaten, signierten Capability Grant.

AccessContext kombiniert die sichere AgentIdentity mit dem erlaubten Projektkontext. Der Ausdruck request.project_id.clone().or_else(|| agent.project_id.clone()) verwendet ein angefragtes Projekt nur, wenn

es vorhanden ist, sonst die serverseitige Projektbindung des Agenten. Eine echte Permission-Pruefung entscheidet danach, ob die Memory passt.

Die for-Schleife ist die Invariante in Codeform: can_read_memory kommt vor score_memory. Nur erlaubte Memory wird zur Bewertung hinzugefuegt. Verbotene Memory erhoeht lediglich einen Zaehler fuer das Audit. Ihr Inhalt wird nicht gerankt oder in den Pack geschoben.

pack_context(scored, request.token_budget) macht aus sortierten Kandidaten einen begrenzten Context Pack. Danach baut der Code ein AuditRecord mit erlaubten IDs, Anzahl abgelehnter Eintraege und Token-Zahlen. Der Audit ist nicht nur Logging - er ist eine fachliche Erklaerung der Entscheidung.

Merksatz

Ein Retrieval-System entscheidet zuerst: Darf ich es sehen? Dann: Ist es relevant? Dann: Passt es ins Budget? Diese Reihenfolge schuetzt Daten und macht Entscheidungen erklaerbar.

22. Embeddings: Text wird zu Suchgeometrie

SOURCE crates/energon-api/src/embedding.rs / 79-112

```
    /// Embed a single query string. Failures are surfaced as errors so callers
    /// can fall back to recency-based retrieval instead of failing requests.
    pub async fn embed(&self, input: &str) -> Result<Vec<f32>, EmbeddingError> {
        let response = self
            .client
            .post(&self.endpoint)
            .bearer_auth(&self.api_key)
            .json(&EmbeddingRequest {
                model: &self.model,
                input: &[input],
            })
            .send()
            .await
            .map_err(|error| EmbeddingError::Request(error.to_string()))?;

        let status = response.status();
        if !status.is_success() {
            return Err(EmbeddingError::Request(format!(
                "OpenAI embeddings returned HTTP {status}"
            )));
        }

        let body = response
            .json::<EmbeddingResponse>()
            .await
            .map_err(|error| EmbeddingError::Malformed(error.to_string()))?;

        body.data
            .into_iter()
            .next()
            .map(|item| item.embedding)
            .filter(|embedding| !embedding.is_empty())
            .ok_or_else(|| EmbeddingError::Malformed("empty embedding data".to_owned()))
    }
```

async fn embed nimmt einen Text und verspricht entweder Result<Vec<f32>, EmbeddingError>. Vec<f32> ist ein dynamisches Array aus 32-Bit-Gleitkommazahlen - genau der Vektor, der spaeter in pgvector gespeichert oder mit anderen Vektoren verglichen werden kann.

client.post(...).bearer_auth(...).json(...).send().await baut einen HTTP POST an den OpenAI Embeddings-Endpunkt. Das API Secret kommt nur aus dem Server-State. Es darf niemals in einen Browser-Bundle oder Agenten-Prompt wandern.

Nach der Antwort prueft is_success() den HTTP-Status und json::<EmbeddingResponse>() prueft die Antwort gegen die erwartete Rust-Struktur. Das sind zwei verschiedene Schutzschichten: Netzwerkstatus und Datenform.

into_iter().next().filter(...).ok_or_else(...) liest den ersten Vektor, verwirft leere Ergebnisse und macht auch das zu einem klaren Fehler. Der Aufrufer kann bei Embedding-Fehlern auf eine einfachere, zum Beispiel zeitbasierte oder Keyword-Suche zurueckfallen, statt den ganzen Context Request kaputtzumachen.

23. Postgres und pgvector: Datenmodell statt lose Textdateien

SOURCE migrations/0002_memory.sql / 1-24, 41-52

```
CREATE TABLE IF NOT EXISTS memory_entries (
    memory_id TEXT PRIMARY KEY,
    org_id TEXT NOT NULL REFERENCES orgs(org_id) ON DELETE CASCADE,
    scope TEXT NOT NULL CHECK (
        scope IN (
            'open',
            'org',
            'project',
            'role',
            'agent_private',
            'user_private',
            'session'
        )
    ),
    content TEXT NOT NULL,
    tags TEXT[] NOT NULL DEFAULT '{}',
    project_id TEXT REFERENCES projects(project_id) ON DELETE CASCADE,
    role_id TEXT REFERENCES roles(role_id) ON DELETE CASCADE,
    owner_agent_id TEXT REFERENCES agents(agent_id) ON DELETE SET NULL,
    user_id TEXT,
    session_id TEXT,
    source TEXT,
    promoted_from TEXT REFERENCES memory_entries(memory_id) ON DELETE SET NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()

CREATE TABLE IF NOT EXISTS memory_chunks (
    chunk_id TEXT PRIMARY KEY,
    memory_id TEXT NOT NULL REFERENCES memory_entries(memory_id) ON DELETE CASCADE,
    chunk_index INTEGER NOT NULL,
    content TEXT NOT NULL,
    embedding vector(1536),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(memory_id, chunk_index)
);

CREATE INDEX IF NOT EXISTS memory_chunks_embedding_idx
    ON memory_chunks USING hnsw (embedding vector_cosine_ops);
```

CREATE TABLE memory_entries definiert die Haupttabelle. memory_id TEXT PRIMARY KEY macht jede Memory eindeutig. org_id ... REFERENCES orgs ist ein Fremdschluessel: Eine Memory darf nicht zu einer Organisation zeigen, die es nicht gibt. ON DELETE CASCADE bedeutet, dass beim Loeschen einer Organisation ihre untergeordneten Eintraege mit entfernt werden.

scope TEXT NOT NULL CHECK (...) beschraenkt den Scope auf bekannte Varianten. Das ist eine zweite Verteidigung neben Rust-Enums: Auch ein direkter SQL-Fehler kann nicht einfach scope = 'everyone_forever' speichern.

promoted_from speichert die Herkunft einer freigegebenen Memory. Dadurch kann ein Audit nachvollziehen: Diese Projekt-Memory entstand aus jener urspruenglich privaten Memory.

memory_chunks teilt lange Inhalte in kleinere Teile. Jeder Chunk kann einen embedding vector(1536) tragen. Der hnsw-Index mit vector_cosine_ops beschleunigt die Suche nach semantisch aehnlichen Vektoren. Wichtig: Der Index verbessert Geschwindigkeit, ersetzt aber nie die Berechtigungspruefung.

24. SDK: Ergonomie ohne Identitaetsfaelschung

SOURCE packages/sdk-typescript/src/index.ts / 298-348

```
/**
 * A server-side client for one authenticated agent in one swarm. Agent identity
 * is always derived by the control plane from the API key, never from SDK input.
 */
export class Energon {
  readonly memory: {
    remember: (input: RememberInput, options?: RequestOptions) => Promise<AgentMemory>;
    share: (input: ShareMemoryInput, options?: RequestOptions) => Promise<AgentMemory>;
  };

  readonly context: {
    build: (input: BuildContextInput, options?: RequestOptions) => Promise<ContextPack>;
  };

  readonly audit: {
    context: (requestId: string, options?: RequestOptions) => Promise<ContextAudit>;
    promotion: (memoryId: string, options?: RequestOptions) => Promise<PromotionAudit>;
  };

  readonly swarm: {
    runtime: (options?: RequestOptions) => Promise<SwarmRuntime>;
  };

  readonly claims: {
    assert: (input: AssertClaimInput, options?: RequestOptions) => Promise<ClaimAssertion>;
  };

  readonly skills: {
    /** Returns only skill profiles assigned to the authenticated agent. */
    list: (options?: RequestOptions) => Promise<SkillProfile[]>;
    /** Creates a private profile for the authenticated agent only. */
    create: (input: CreateSkillInput, options?: RequestOptions) => Promise<SkillProfile>;
  };

  readonly operations: {
    /** Agent-safe organization state and the dashboard's live operational metrics. */
    overview: (options?: RequestOptions) => Promise<AgentOperationalOverview>;
  };

  private readonly baseUrl: string;
  private readonly apiKey: string;
  private readonly paymentSignature?: PaymentSignatureProvider;
  private readonly timeoutMs: number;
  private readonly fetchFn: typeof fetch;

  constructor(options: EnergonClientOptions) {
    this.baseUrl = normalizeBaseUrl(options.baseUrl);
    this.apiKey = requiredText(options.apiKey, "apiKey");
    this.paymentSignature = options.paymentSignature;
    this.timeoutMs = normalizeTimeout(options.timeoutMs);
    this.fetchFn = options.fetch ?? globalThis.fetch;
```

Der Kommentar ueber der Klasse beschreibt den Vertrag: Ein Energon-Objekt repraesentiert genau einen authentifizierten Agenten. readonly memory, context, audit und weitere Bereiche geben dem Entwickler eine klare, typisierte API.

Die Methoden nehmen fachliche Eingaben wie RememberInput oder BuildContextInput. Auffaellig fehlt: org_id, role_id oder agent_id. Das ist Absicht. Ein SDK-Nutzer darf die Aufgabe und den Inhalt benennen, aber nicht seine Berechtigungsidentitaet behaupten.

Im Konstruktor normalisiert normalizeBaseUrl die Serveradresse, requiredText prueft, ob ein Key existiert, und normalizeTimeout setzt ein erwartbares Zeitlimit. Das sind kleine Details, die Integrationen robust machen und Fehler frueh sichtbar machen.

TypeScript hilft hier vor allem dem SDK-Nutzer. Die letzte Sicherheitsentscheidung bleibt trotzdem im Rust-Backend, weil jeder Client theoretisch veraendert oder umgangen werden kann.

25. Worker: langsame Nebenarbeit entkoppeln

SOURCE crates/energon-worker/src/main.rs / 63-83

```
    loop {
        let embedded = match &openai_api_key {
            Some(api_key) => {
                process_embedding_batch(&pool, &client, api_key, &model, batch_size).await?
            }
            None => 0,
        };
        let published = match &publisher {
            Some(publisher) => publish_outbox_batch(&pool, publisher, event_batch_size).await?,
            None => 0,
        };

        if run_once {
            tracing::info!(embedded, published, "worker one-shot finished");
            break;
        }

        if embedded == 0 && published == 0 {
            tokio::time::sleep(Duration::from_secs(2)).await;
        }
    }
```

Die loop ist eine dauerhafte Worker-Schleife. Wenn ein OpenAI-Key konfiguriert ist, verarbeitet sie eine kleine Embedding-Batch. Wenn ein NATS Publisher konfiguriert ist, veroeffentlicht sie eine Outbox-Batch. Beide Ergebnisse werden als Zahlen gespeichert: embedded und published.

run_once ist eine kontrollierte Betriebsoption. In Tests, Jobs oder beim manuellen Debugging darf der Worker nach einem Durchlauf sauber enden. In Produktion laeuft er weiter.

Wenn weder Embeddings noch Events Arbeit fanden, wartet tokio::time::sleep zwei Sekunden. Das verhindert eine enge CPU-Schleife, die ununterbrochen dieselbe leere Datenbank abfragt. Weil sleep awaited wird, blockiert er nicht den ganzen Tokio-Thread.

End-to-end: eine echte Context-Anfrage

1. Agent	Ein Agent ruft context.build im SDK auf. Der Key reist im Authorization Header mit.
2. API Edge	Axum Router, Limits und CORS nehmen die Anfrage an. Auth hasht den Key und laedt die AgentIdentity.
3. Daten	Postgres liefert begrenzte Kandidaten fuer Organisation, Projekt, Rolle und Besitzer. Ein Vektorindex kann Relevanz beschleunigen.
4. Core	can_read_memory filtert. Erst danach wird bewertet, sortiert und innerhalb des Token Budgets gepackt.
5. Audit	Erlaubte IDs, abgelehnte Anzahl, Aufgabe und Token-Zahlen werden gespeichert. Der Agent erhaelt nur seinen Pack.

26. Was du als Naechstes lernen solltest

Diese Reihenfolge ist absichtlich praktisch. Nach jedem Schritt solltest du nicht nur lesen, sondern eine kleine Frage am echten Code beantworten.

Woche 1	Rust-Basics: Variablen, Structs, Enums, Match, Option, Result, Vec. Lies energon-core/src/memory.rs und identity.rs.
Woche 2	Ownership und async: Erklaere jedes &, clone() und .await im Context Broker mit eigenen Worten.
Woche 3	HTTP und Axum: Folge einer Route von app.rs bis zum Handler. Teste einen lokalen Request mit absichtlich falschem Key.
Woche 4	SQL und Datenmodell: Lies die Migrations. Zeichne Memory, Agent, Projekt und Audit als Beziehungen auf Papier.
Woche 5	AI Retrieval: Schreibe fuer dich auf, warum Embedding-Aehnlichkeit nie eine Berechtigung ist. Vergleiche Keyword- und Vektorsuche.
Woche 6	Betrieb und Security: Untersuche Environment Variables, Logs, Rate Limits, Outbox und die Fehlerfaelle eines Worker-Restarts.

Abschluss

Du musst nicht jede Zeile sofort verstehen. Das Ziel ist ein belastbares Modell: Energon ist eine Rust-Control-Plane, die Identitaet serverseitig ableitet, Memory nach Berechtigung filtert, AI Retrieval nur fuer erlaubte Kandidaten nutzt und jede Context-Entscheidung pruefbar macht.

> Wenn du bei einer neuen Datei feststeckst, stelle immer dieselben Fragen: Welche Daten kommen hinein? Welcher Typ repraesentiert sie? Welche Schicht besitzt die Entscheidung? Was passiert bei Fehlern? Welche Invariante darf nie gebrochen werden? Damit liest du dich Schritt fuer Schritt zu echtem Senior-Systemverstaendnis vor.